⚙️ Phase 2: Local Environment Setup

Once software is installed, open your terminal (PowerShell on Windows, Terminal on Mac/Linux) and run these steps to set up your workspace:

1. Clone the Repository

Download the project code to your machine:

PowerShell

```
git clone https://github.com/YOUR_USERNAME/trialguard.git
cd trialguard
```

(Replace `YOUR_USERNAME` with the main repository owner's username).

2. Create a Local Virtual Environment

Isolate your project packages so they don't conflict with other Python projects:

PowerShell

```
python -m venv .venv
```

3. Activate the Virtual Environment

Windows (PowerShell):

```
PowerShell

.venv\Scripts\Activate.ps1
```

Mac / Linux:

```
Bash

source .venv/bin/activate
```

4. Install Project Dependencies

Install all required libraries (FastAPI, Streamlit, XGBoost, Scikit-Learn, NetworkX, Pandas, Pytest):

```
PowerShell

pip install -r requirements.txt
```

5. Verify the Installation

Run the test suite to confirm everything is working correctly on your machine:

```
PowerShell

python -m pytest
```

🔀 Phase 3: Git Collaboration & Anti-Overlap Workflow

To prevent merge conflicts and ensure nobody overwrites each other's code, follow this strict team workflow:

1. Clear Division of Responsibilities

To avoid editing the same files, divide your hands-on tasks cleanly:

- **Team Member 1 (Cloud Mirror & UI Portals):** Primarily responsible for frontend interfaces (e.g., `app/cloud_console.py` or mock portal views).
- **Team Member 2 (Security HQ & Backend Engine):** Primarily responsible for backend logic (`src/`) and analytics views (`app/security_hq.py`).

2. Daily Workflow: Always Pull Before You Code

Whenever you sit down at your computer to work, **always pull the latest changes** from GitHub before writing any code:

PowerShell

```
git pull origin main
```

3. How to Commit and Push Your Work Safely

When you finish a task or feature, push it to GitHub using these 4 steps:

Step A: Check modified files

PowerShell

```
git status
```

Step B: Stage your changes

PowerShell

```
git add .
```

Step C: Commit with a clear message

PowerShell

```
git commit -m "Brief description of what you built or fixed"
```

Step D: Push to GitHub

PowerShell

```
git push origin main
```

4. Handling Merge Conflicts (If They Happen)

If you and your teammate accidentally modify the same line of code, Git will warn you during a pull or push.

Run `git pull origin main`.

Open the conflicting file in VS Code. Look for the conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`).

Choose whether to keep Incoming Changes, Current Changes, or Both.

Save the file, stage it (`git add .`), commit (`git commit -m "Resolve merge conflict"`), and push (`git push origin main`).